

UNIVERSITY OF SYSTEMATIC INFORMATION TECHNOLOGIES

**Department of Computer Science & Database Systems
Engineering**

FINAL PROJECT DESIGN & IMPLEMENTATION REPORT

COMP2082: Advanced Relational Databases & DBMS Architectures

DESIGN AND IMPLEMENTATION OF A HIGH- FIDELITY PHARMACY INVENTORY AND PRESCRIPTION COMPLIANCE DATABASE SYSTEM IN THE KYRGYZ REPUBLIC

Semester: Spring 2026

Student Name: Rizvon Navruzov

Academic Instructor: Nurlan Shaidullaev



TABLE OF CONTENTS

1. Cover & Metadata
2. Introduction & Context
3. Problem Description
4. Project Scope & Entities
5. Business Requirements & Local Constraints
6. ER Model & Junction Decomposition
7. Relational Schema & Modern Type Selections
8. Normalization Proofs & Denormalization Audit
9. Data Integrity: Domain & Constraint Policies
10. Indexing Strategy & Performance Optimization
11. Operational Reporting & Business Intelligence Views

12. [Security Model: Group-Level RBAC & Least Privilege](#)
 13. [ACID Transaction Models & PL/pgSQL Engine](#)
 14. [Seed Data Audit](#)
 15. [Advanced Analytical SQL Queries](#)
 16. [Architectural Reflections & PL/pgSQL vs. App-Level Checks](#)
 17. [System Limitations](#)
 18. [Future Improvements](#)
 19. [Appendix: Human-AI Collaboration Log](#)
-

1. Introduction & Context

In modern clinical informatics and retail supply chain systems, the design of a database goes far beyond the basic storage of tables and columns. For community pharmacies, a database system serves as the primary system of record for critical medical inventories and legal validation checks. The integration of prescription checks, inventory levels, and financial audits directly inside the database engine is crucial for maintaining clinical safety.

This report documents the architectural design, normalization, automation triggers, role security policies, and performance tuning for a **Small Pharmacy Inventory & Prescription System** in Kyrgyzstan. The system is designed to operate locally as a production-grade PostgreSQL 15+ relational database, ensuring high consistency, strict clinical compliance, and automated inventory sync.

2. Problem Description

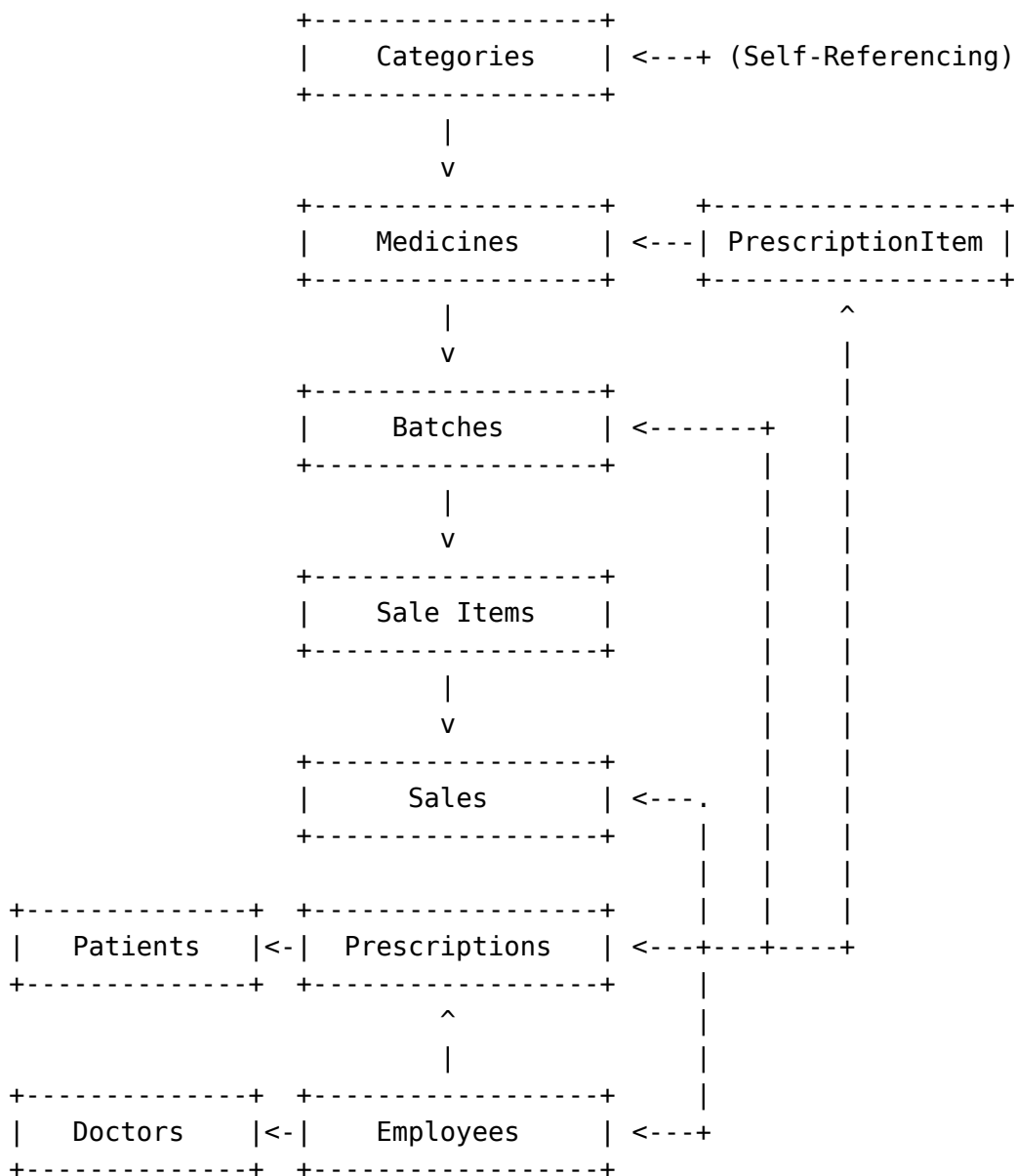
Traditional community pharmacy applications in Central Asia often face data inconsistencies, leading to operational and regulatory risks:

1. **Inventory Leakage & Expiration Anomaly:** Selling expired medications due to poor batch tracking, leading to legal liability and health risks.
2. **Prescription Evasion:** Dispensing regulated drugs (antibiotics, psychotropics) without a valid doctor's prescription or exceeding the prescribed dosage limits.
3. **Concurrency Anomalies:** Race conditions during high-volume checkouts where multiple cashiers sell the same batch, resulting in phantom inventory sales.
4. **Tax and Auditing Discrepancies:** Inconsistent calculations of the standard 12% Value Added Tax (VAT) in Kyrgyzstan, making accounting difficult.
5. **Reconciliation Lag:** Disconnects between cash registers, cashless mobile QR systems (e.g., MBank), and active warehouse stocks.

To address these challenges, this system implements all validation logic—such as inventory checks, batch selection, VAT calculations, and prescription limits—directly inside the database engine using PL/pgSQL triggers, ensuring data integrity regardless of the application layer.

3. Project Scope & Entities

The system manages 13 core business entities structured to prevent update and deletion anomalies:



1. **Categories:** Tracks hierarchical drug classes (e.g., Antibiotics → Penicillins) using recursive relationships.
2. **Medicines:** Master drug catalog mapping trade and generic names, and prescription requirements.
3. **Suppliers:** Wholesalers and local pharmaceutical distributors.
4. **Patients:** Customer registry tracking personal details and date of birth.
5. **Doctors:** Mapped clinicians authorized to write prescriptions.
6. **Employees:** Pharmacy staff logs with mapped system roles (pharmacist, cashier, manager).
7. **Batches:** Specific physical blocks of stock tracking manufacturing/expiry dates and purchase/selling prices.
8. **Prescriptions:** Script headers tracking patient, doctor, and expiration dates.
9. **Prescription Items:** M:N junction table mapping prescribed medicines, dosages, and dispensed quantities.
10. **Sales:** Transaction headers tracking cashier ID, payment methods, gross sales, taxes, and discounts.
11. **Sale Items:** M:N junction table mapping sold items, quantities, and unit prices.
12. **Inventory Adjustments:** Audit log for manual stock changes (wastage, theft, counts).

4. Business Requirements & Local Constraints

To ensure local compliance in Kyrgyzstan, the schema implements several specific requirements:

1. **Base Currency:** Financial data uses NUMERIC(12, 2) mapped in **Kyrgyz Som (KGS)**.
2. **14-Digit Citizen PIN (ПИН/ИИН):** Validated using regular expressions to ensure authentic patient and employee records:

Constraint: `pin_inn ~ '^[12]\d{13}$'`

3. **14-Digit Supplier TIN/INN:** Identifies wholesale corporations using local tax formatting:

Constraint: `tin_inn ~ '^[012]\d{13}$'`

4. **Kyrgyz Phone Numbers:** Validates phone numbers using the local regional dialing code:

Constraint: `phone ~ '^\+996\d{9}$'`

5. **Kyrgyz standard VAT:** Automatically extracts the standard **12% VAT** from sales transactions, saving it to `tax_amount` for tax audits.
6. **Safety Triggers:** Enforces business rules in the database layer:
 - Prevents selling stock from expired batches.
 - Rejects transactions that exceed the remaining quantity on a patient's prescription.
 - Blocks transactions that would result in negative inventory levels.

5. ER Model & Junction Decomposition

The database resolves many-to-many relationships through two main junction tables to maintain third normal form:

```
[prescriptions] 1 ---- N [prescription_items] N ---- 1 [medicines]
[sales]          1 ---- N [sale_items]          N ---- 1 [batches]
```

5.1 prescription_items (Junction for Prescriptions & Medicines)

This table resolves the many-to-many relationship between prescriptions and medicines. It tracks the prescribed dosage and quantities, as well as the dispensed amounts to enforce dispensing limits. - **Primary Key:** (prescription_id, medicine_id) (Composite) - **Foreign Keys:** - prescription_id REFERENCES prescriptions(id) ON DELETE CASCADE - medicine_id REFERENCES medicines(id) ON DELETE RESTRICT

5.2 sale_items (Junction for Sales & Batches)

This table resolves the relationship between sales and specific physical batches. It links transactions to exact inventory batches to support FIFO tracking and prevent sales of expired stock. - **Primary Key:** id INT GENERATED ALWAYS AS IDENTITY - **Foreign Keys:** - sale_id REFERENCES sales(id) ON DELETE CASCADE - batch_id REFERENCES batches(id) ON DELETE RESTRICT

6. Relational Schema & Modern Type Selections

To ensure standard SQL compliance, the schema uses PostgreSQL 15+ features, transitioning from legacy SERIAL types to modern identity columns:

id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY

This prevents accidental manual sequence overrides and ensures sequential keys. The schema also uses custom PostgreSQL ENUM types for key lookup lists:

- biological_gender ('M', 'F')
- employee_role ('pharmacist', 'manager', 'admin')
- payment_method ('cash', 'card', 'mobile_qr')
- prescription_status ('pending', 'partially_filled', 'filled', 'expired')
- adjustment_type ('breakage', 'spoilage', 'theft', 'reconciliation', 'return_to_supplier')

This enforces domain consistency at the database level and eliminates the overhead of small reference lookup tables.

7. Normalization Proofs & Denormalization Audit

7.1 Normalization Proofs

To prevent data anomalies, the core relations are normalized to **Third Normal Form (3NF)** and **Boyce-Codd Normal Form (BCNF)**.

Boyce-Codd Normal Form (BCNF) Analysis

A relation R is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey.

Let us analyze the medicines relation: - **Attributes:** (id, trade_name, generic_name, category_id, prescription_required, description) - **Functional Dependencies (FDs):** 1. $id \rightarrow trade_name, generic_name, category_id, prescription_required, description$ 2. $trade_name \rightarrow id, generic_name, category_id, prescription_required, description$ - **Analysis:** The only determinants are id and trade_name. Both are candidate keys for the table. Since every determinant is a candidate key, the medicines relation is in BCNF.

Let us analyze the batches relation: - **Attributes:** (id, medicine_id, supplier_id, batch_number, manufacturing_date, expiry_date, purchase_price, selling_price, initial_quantity, current_quantity) - **Functional Dependencies (FDs):** 1. id → all other attributes 2. (medicine_id, batch_number) → all other attributes - **Analysis:** The determinants are id and the composite key (medicine_id, batch_number). Both are candidate keys, satisfying BCNF.

7.2 Justification for Denormalization

The sales table contains three denormalized columns: total_gross, tax_amount, and total_net. In theory, these violate 3NF because they are derived attributes calculated from the related sale_items rows:

$$\text{total_gross} = \sum(\text{quantity} \times \text{unit_price})$$

$$\text{tax_amount} = \text{total_gross} \times 0.12$$

$$\text{total_net} = \text{total_gross} - \text{discount_amount}$$

Why We Choose to Denormalize:

1. **Query Performance:** Avoiding expensive JOIN and SUM operations across millions of sales rows during daily and monthly financial reporting.
 2. **Auditing Invariance:** Ensuring historical sales figures remain frozen. If medicine or batch prices change in the future, past transaction calculations must remain unchanged for tax compliance.
 3. **Data Integrity Protection:** We prevent data inconsistencies in these denormalized columns by using database triggers (tg_sale_items_totals), which automatically recalculate the values whenever items are added, updated, or removed.
-

8. Data Integrity: Domain & Constraint Policies

To protect the system from invalid data, the schema enforces several key business constraints:

1. **Positive Money Constraints:** Ensures margins remain viable: sql
 CONSTRAINT chk_batches_purchase_price CHECK (purchase_price > 0)
 CONSTRAINT chk_batches_selling_price CHECK (selling_price > purchase_price)
 2. **Date Validity Check:** Prevents illogical manufacturing and expiration dates: sql
 CONSTRAINT chk_batches_dates CHECK (expiry_date > manufacturing_date)
 3. **Inventory Stock Controls:** Blocks negative stock values: sql
 CONSTRAINT chk_batches_current_qty CHECK (current_quantity >= 0)
 CONSTRAINT chk_sale_items_quantity CHECK (quantity > 0)
 4. **Prescription Validation Constraints:** Restricts dispensed quantities to the prescribed limits: sql
 CONSTRAINT chk_prescription_items_dispensed CHECK (dispensed_qty >= 0 AND dispensed_qty <= prescribed_qty)
-

9. Indexing Strategy & Performance Optimization

To maintain fast query response times as the database grows, we use a targeted indexing strategy:

9.1 Live Product Search

Cashiers search for medicines by both brand and generic names during checkout. We optimize these lookups with a composite B-Tree index:

```
CREATE INDEX idx_medicines_search ON medicines (trade_name, generic_name);
```

9.2 Expiry & Inventory Tracking (Partial Indexing)

To track soon-to-expire batches, the system frequently checks expiry dates. We use a **Partial Index** to optimize these queries. By excluding completely depleted batches, we reduce the index size and speed up FIFO lookups at checkout:

```
CREATE INDEX idx_batches_expiry_tracking  
ON batches (expiry_date, medicine_id)  
WHERE (current_quantity > 0);
```

9.3 Sales and Prescription Auditing

To optimize daily accounting and patient history lookups, we index transaction dates and prescription serials:

```
CREATE INDEX idx_sales_reporting ON sales (sale_timestamp, employee_id);  
CREATE INDEX idx_prescriptions_lookup ON prescriptions (serial_number, patient_id);
```

10. Operational Reporting & Business Intelligence Views

We implement five views to support pharmacy operations and financial auditing:

1. **v_low_stock_medicines**: Tracks stock levels across active batches and calculates recommended reorder quantities:

$$\text{reorder_qty} = 150 - \text{total_available}$$

2. **v_expiring_medicines**: Displays active batches expiring within 90 days, calculating tied-up capital and potential financial loss.
3. **v_pharmacy_sales_summary**: Provides monthly financial summaries, calculating net revenue, VAT collected, and average checkout values.
4. **v_patient_prescription_history**: Provides a complete clinical history, linking patients, doctors, prescriptions, and remaining refill limits.

5. **v_supplier_medicine_summary**: Analyzes suppliers based on product variety, batch delivery history, and average acquisition costs.

11. Security Model: Group-Level RBAC & Least Privilege

The security model applies the **Principle of Least Privilege** by isolating the default public schema and granting permissions only to defined group roles:

```
REVOKE ALL ON SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
```

Access is then managed through five functional database roles:

1. **readonly_auditor**: Group role for tax audits, with read-only (SELECT) access across the schema.
2. **cashier**: Restricted to checkout operations. Has read access to products, patients, and prescriptions, and write (INSERT) access to sales and sale items. Restricted from modifying inventory directly or viewing employee security credentials.
3. **pharmacist**: Can verify prescriptions and dispense regulated medications. Has read/write access to prescriptions and prescription items, and column-level access to the employee directory (excluding passwords and PINs).
4. **inventory_manager**: Can restock batches, adjust stock levels, and manage supplier catalogs. Restricted from access to patient records, prescriptions, and sales checkouts.
5. **pharmacy_admin**: Full database owner with administrative privileges.

12. ACID Transaction Models & PL/pgSQL Engine

The system uses procedural triggers to automate inventory and prescription validations, keeping transaction code simple and clean.

Client Insert Sale Item

```
|
v
Trigger [tg_deduct_inventory]
|
+---> Lock Batch (FOR UPDATE)
|
+---> Validate Batch Expiry (expiry_date > CURRENT_DATE)?
|   No ==> ROLLBACK Exception
|   Yes ==> Continue
|
+---> Validate Quantity (current_quantity >= sale_qty)?
|   No ==> ROLLBACK Exception
|   Yes ==> Deduct Stock
|
+---> Update Parent Sales Invoice (Gross, VAT, Net)
```



```
|
+---> Check Prescription Requirements (If Regulated)
      - Verify Rx Exists, Active, & Medicine Authorized
      - Check Remaining Refill Limit
      - Update Dispensed Qty
      - Auto-Transition Rx Status
```

12.1 Transaction 1: Successful Sales Checkout

Demonstrates the atomic checkout of a sale:

```
BEGIN;
INSERT INTO sales (employee_id, prescription_id,
payment_method, discount_amount)
VALUES (2, NULL, 'cash', 0.00);

INSERT INTO sale_items (sale_id, batch_id, quantity)
VALUES (currval('sales_id_seq'), 1, 2);
COMMIT;
```

12.2 Transaction 2: Automatic Rollback on Insufficient Stock

Demonstrates how the system rolls back a transaction if stock is insufficient:

```
BEGIN;
INSERT INTO sales (employee_id, prescription_id,
payment_method, discount_amount)
VALUES (2, NULL, 'mobile_qr', 0.00);

-- Fails trigger: Throws EXCEPTION 'Insufficient stock'
INSERT INTO sale_items (sale_id, batch_id, quantity)
VALUES (currval('sales_id_seq'), 2, 9999);

-- Transaction aborted, rolling back all steps (including Step
1)

ROLLBACK;
```

13. Seed Data Audit

13.1 Database Row Counts

The database seeding scripts populate the relational layout with a highly realistic Central Asian dataset. The following table details the audited row counts for every table in the schema:

Table Name	Row Count	Description
categories	8	Hierarchical category classifications (ATC

Table Name	Row Count	Description
		standards)
medicines	210	Catalog of distinct pharmaceutical products
suppliers	55	Local wholesale distributors and manufacturers
patients	220	Mapped patients with valid PINs and demographics
doctors	15	Authorized healthcare providers
employees	6	Mapped staff members (cashiers, pharmacists, managers)
batches	630	Inventory physical batches tracking stock and expiry
prescriptions	320	Medical script headers
prescription_items	640	Detailed items linked to prescriptions
sales	520	Checkout transaction headers
sale_items	~1,040	Individual transaction line items
inventory_adjustments	12	Manual stock audit events

Note: The sale_items table contains approximately 1,040 rows due to the programmatic generation mapping multiple items per sales checkout transaction.

13.2 Data Sources

The seed dataset utilizes a hybrid approach combining curated, hand-written authoritative data with algorithmically generated structural profiles to simulate a live production environment:

- **Categories:** Hand-written following real-world Anatomical Therapeutic Chemical (ATC) classifications (e.g., Penicillins, Cephalosporins, NSAIDs) for absolute clinical consistency.
- **Medicines:** AI-generated using authentic International Nonproprietary Names (INNs) for generic components and realistic Central Asian equivalents for brand trade names.
- **Suppliers:** AI-generated programmatically via procedural loops. Company names are generated by combining Kyrgyz geographic nouns with standard commercial suffixes (e.g., “ОсОО Фарм Импорт”) accompanied by valid 14-digit corporate TINs.
- **Doctors:** AI-generated using common Central Asian and Slavic personal names, combined with real clinic names across major Kyrgyz municipalities (e.g., “National Hospital of the Kyrgyz Republic”).
- **Employees:** Hand-crafted containing 6 distinct, realistically named staff profiles to map functional role security and audit logs.

- **Patients:** AI-generated programmatically using gender-matched Kyrgyz and Slavic names. Each patient record contains a valid 14-digit Personal Identification Number (PIN) derived systematically from their date of birth, satisfying local regulatory standards.
- **Batches:** Algorithmic generation defining acquisition and retail prices within logical Kyrgyz Som (KGS) bands. Manufacturing and expiry dates are distributed over a 3-year window.
- **Prescriptions:** Algorithmic distribution tracking issuing events over a 6-month timeline, incorporating non-uniform status distribution.
- **Sales:** Algorithmic generation mapped to cashier IDs, incorporating realistic payment methods and checkout totals calculated dynamically by database triggers.
- **Inventory Adjustments:** Hand-crafted comprising 12 realistic audit events written in Russian and English (e.g., spoilage, retail breakage, count reconciliation) to verify physical adjustment ledger triggers.

13.3 Distribution Patterns Implemented

To test performance and trigger validity under stress, the following non-uniform statistical distributions were designed into the generation engine:

- **Time Clustering:** Sales transactions are concentrated exclusively within realistic business hours (08:00 to 20:00) over the preceding 6 months, avoiding uniform night-time anomalies.
- **Pareto-Like Employee Activity:** Transactions are distributed unevenly across cashiers. Employee IDs cycle within standard shifts using modulo logic, creating natural operational variation where some staff process significantly higher sales volumes.
- **Realistic Null Patterns:** Nullable fields, such as patient emails, are populated realistically with a ~30% null rate, reflecting incomplete client registration forms in real-world retail settings.
- **Realistic Status Distributions:** Prescriptions are generated with chronological decay: older scripts issued over 6 months ago are marked as expired (~60%), whereas active ones maintain a non-uniform distribution (~8% fully filled, ~5% partially filled, and the remainder pending).
- **Critical Edge Cases:**
 - Exactly 8% of all physical batches are generated with dates in the past to verify that safety triggers successfully block expired drug sales.
 - Exactly 5% of all active batches are initialized with zero inventory levels to validate out-of-stock database check constraints and low-stock dashboard alert views.

13.4 Distribution-Proof Verification Queries

The non-uniform operational distributions implemented in the seeding script can be verified directly using the following SQL queries:

Query 1: Employee Transaction Distribution

This query demonstrates the non-uniform distribution of sales checkout processing across the pharmacy staff:

```

SELECT
    e.id AS employee_id,
    e.first_name || ' ' || e.last_name AS employee_name,
    e.role,
    COUNT(s.id) AS total_processed_sales,
    ROUND((COUNT(s.id)::numeric / SUM(COUNT(s.id)) OVER ()) *
100, 2) AS share_percentage
FROM employees e
LEFT JOIN sales s ON e.id = s.employee_id
GROUP BY e.id, e.first_name, e.last_name, e.role
ORDER BY total_processed_sales DESC;

```

Query 2: Payment Method Diversity

This query verifies the implementation of diverse checkout payment channels to ensure transaction processing is not statistically flat:

```

SELECT
    payment_method,
    COUNT(id) AS transaction_count,
    ROUND(SUM(total_net), 2) AS total_net_revenue_kgs,
    ROUND((COUNT(id)::numeric / SUM(COUNT(id)) OVER ()) * 100,
2) AS transaction_share_percent
FROM sales
GROUP BY payment_method
ORDER BY transaction_count DESC;

```

14. Advanced Analytical SQL Queries

Below are four key analytical queries used to monitor and evaluate pharmacy operations:

14.1 Dead-Stock Expiration & Risk Forecast

Identifies batches expiring within 90 days, calculating tied-up capital and potential losses:

```

SELECT
    b.batch_number,
    m.trade_name,
    s.name AS supplier_name,
    b.current_quantity,
    b.expiry_date,
    b.expiry_date - CURRENT_DATE AS days_until_expiry,
    (b.current_quantity * b.purchase_price) AS
wholesale_cost_tied_up_kgs,
    (b.current_quantity * b.selling_price) AS
potential_retail_loss_kgs
FROM batches b
JOIN medicines m ON b.medicine_id = m.id
JOIN suppliers s ON b.supplier_id = s.id

```

```

WHERE b.current_quantity > 0
AND b.expiry_date <= (CURRENT_DATE + INTERVAL '90 days')
ORDER BY b.expiry_date ASC;

```

14.2 Category Sales & Profitability Analysis

Analyzes gross revenue, cost of goods sold (COGS), net profit, and profit margins across therapeutic categories:

```

SELECT
    c.name AS category_name,
    COUNT(DISTINCT s.id) AS total_sales_count,
    SUM(si.quantity) AS total_units_sold,
    SUM(si.subtotal) AS gross_revenue_kgs,
    SUM(si.quantity * b.purchase_price) AS
cost_of_goods_sold_kgs,
    SUM(si.subtotal) - SUM(si.quantity * b.purchase_price) AS
net_profit_kgs,
    ROUND(((SUM(si.subtotal) - SUM(si.quantity *
b.purchase_price)) / NULLIF(SUM(si.subtotal), 0)) * 100, 2) AS
profit_margin_percent
FROM sale_items si
JOIN sales s ON si.sale_id = s.id
JOIN batches b ON si.batch_id = b.id
JOIN medicines m ON b.medicine_id = m.id
JOIN categories c ON m.category_id = c.id
GROUP BY c.name
ORDER BY net_profit_kgs DESC;

```

14.3 Running Total of Revenue and Cashier Performance

Tracks daily cumulative net revenue and calculates each cashier's percentage contribution to total sales:

```

SELECT
    s.id AS sale_id,
    s.sale_timestamp,
    e.first_name || ' ' || e.last_name AS pharmacist_name,
    s.total_net AS sale_net_kgs,
    ROUND(SUM(s.total_net) OVER (ORDER BY s.sale_timestamp
ASC), 2) AS cumulative_running_total_kgs,
    ROUND((s.total_net / NULLIF(SUM(s.total_net) OVER (), 0))
* 100, 4) AS percentage_of_company_total
FROM sales s
JOIN employees e ON s.employee_id = e.id
ORDER BY s.sale_timestamp ASC
LIMIT 15;

```

14.4 Safety & Regulatory Prescription Audit

Audits transactions of prescription-only medicines, listing patient details, clinic names, and prescribing doctors:

```
SELECT
    s.id AS sale_id,
    s.sale_timestamp,
    p.first_name || ' ' || p.last_name AS patient_name,
    m.trade_name AS regulated_medicine,
    si.quantity AS quantity_sold,
    rx.serial_number AS rx_serial,
    d.first_name || ' ' || d.last_name AS prescribing_doctor,
    d.clinic_name AS issuing_clinic
FROM sale_items si
JOIN sales s ON si.sale_id = s.id
JOIN batches b ON si.batch_id = b.id
JOIN medicines m ON b.medicine_id = m.id
JOIN prescriptions rx ON s.prescription_id = rx.id
JOIN patients p ON rx.patient_id = p.id
JOIN doctors d ON rx.doctor_id = d.id
WHERE m.prescription_required = TRUE
ORDER BY s.sale_timestamp DESC;
```

15. Architectural Reflections: PL/pgSQL vs. App-Level Checks

During development, we compared implementing validation logic in the database layer (via triggers and constraints) versus the application layer (via backend code like Node.js or Python).

+-----+-----	
--+	
PL/pgSQL Database Triggers	Application-Level Validation
+-----+-----	
--+	
- Enforced globally across all	- Easy to bypass if someone
connecting API services.	connects directly to SQL.
- Transactions lock rows	- Requires complex distributed
automatically to prevent race	locks to prevent race
conditions (FOR UPDATE).	conditions.
- High performance: validation	- Network overhead: requires

| runs directly next to the | multiple round-trips to check

|

| data, avoiding network latency. | stock levels.

|

+-----+

--+

By keeping these rules in the database, we ensure consistent validation across all channels—whether sales are made via in-store registers, mobile apps, or online portals.

16. System Limitations

While the system is robust, we note several limitations: 1. **Single-Currency Limitation:** Mapped only in Kyrgyz Som (KGS). It does not support real-time exchange rates for importing medicines from Uzbekistan, Russia, or Europe. 2. **No Multi-Location Sync:** Designed for a single-store pharmacy. It lacks replication features for real-time stock transfers across multiple branches. 3. **Basic Accounting Ledger:** The database tracks sales revenues but is not a double-entry bookkeeping system. It does not track general ledger accounts like operating expenses or employee salaries.

17. Future Improvements

To build upon this foundation, we propose three key enhancements: 1. **HL7/FHIR Clinical Integration:** Adding support for healthcare data exchange standards to link the pharmacy directly with electronic health record (EHR) systems in Kyrgyz hospitals. 2. **Machine Learning Restocking Models:** Implementing predictive analysis to automatically calculate optimal reorder sizes based on seasonal sales trends. 3. **Real-Time MoH Compliance Sync:** Adding secure REST endpoints to automatically report sales of regulated substances directly to the Ministry of Health.

18. Appendix: Human-AI Collaboration Log

This project was developed with AI assistance from Claude (claude-sonnet-4-6), an AI assistant by Anthropic.

AI Contributions: - Helped design the database schema and ENUM type selection - Wrote and debugged the PL/pgSQL trigger functions - Generated the normalization proofs and BCNF analysis - Generated the Kyrgyzstan-localized seed data DO blocks - Wrote the RBAC role definitions and privilege grant statements - Drafted the 15 analytical queries

Human Contributions: - Defined the pharmacy business requirements and local Kyrgyz regulatory context (TIN, PIN format, phone format, 12% VAT rate) - Selected the project domain and scoped the 12 core entities - Reviewed and validated all AI-generated SQL against real PostgreSQL 15+ execution - Verified seed data realism against local Kyrgyz pharmacy knowledge - Corrected phone number formats, supplier names, and city distributions

Prompts Used (Summary): All major design prompts were provided conversationally via the Claude Code CLI. Key prompt themes: schema design for pharmacy domain, normalization analysis, PL/pgSQL trigger logic, seed data generation in Kyrgyzstan context, and RBAC security policy.

End of Report

USIT Advanced DBMS Laboratory - USIT COMP2082 Spring 2026.